

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	40% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Madhavan S	Reg. No.:	192425113
Section / Batch:	2024 CSE AI	Date:	18/08/2026

Code of Conduct

I Madhavan s/192425113 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Madhavan S _____

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Professionalism	Sub. Total	Total %
1								
2								
3								

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. CFG Trees vs. Dependency Parsing – Representing Sentence Structure

Scenario: A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.

Working Principles

CFG (constituency) trees:

Build a hierarchical phrase-structure tree using non-terminal category nodes (S, NP, VP, PP ...) and rewrite rules (e.g., $S \rightarrow NP VP$, $VP \rightarrow V NP$). Every word is a leaf, grouped bottom-up into successively larger constituents. The relationship between two words is only implicit — it must be inferred from how many non-terminal nodes separate them in the tree.

Dependency parsing:

Represents the sentence as a graph of exactly one node per word plus one root, connected by directed, labeled edges (e.g., *nsubj*, *dobj*, *amod*) that go from a head word to its dependent. There are no phrasal non-terminal nodes at all — every edge already is a labeled grammatical relationship between two specific words.

Comparison

Aspect	CFG (Constituency) Trees	Dependency Parsing
Basic unit	Phrases/constituents (NP, VP, PP) plus words	Words only – one node per word
Number of nodes/edges	Words + non-terminal nodes (larger tree)	Exactly $n-1$ edges for n words – compact
Word-to-word relations	Implicit; must be derived via head-finding rules	Explicit and labeled on each edge (subject, object, modifier, ...)
Captures phrase boundaries	Yes – directly (constituency is the point)	No – phrases are not first-class objects
Free / flexible word order	Needs many extra rules to cover reorderings	Handles it naturally since relations, not linear position, define structure
Best suited for	Checking whether a phrase is well-formed (grammaticality of constituents)	Extracting who-did-what-to-whom, agreement, argument structure

Justification – Which Is More Effective for Word Relationships

For the specific goal of capturing relationships between words, dependency parsing is the more effective representation:

- **Directness:** a dependency edge such as *nsubj*(saw, She) states the subject relation explicitly; the same fact in a CFG tree must be recovered indirectly by finding the NP under S and applying head rules to identify which word is the “real” subject.
- **Compactness and efficiency:** with only $n-1$ edges for n words, dependency structures are simpler to build, store, and query than a full constituent tree with extra non-terminal nodes.
- **Direct support for grammar checking of relations:** subject–verb agreement, verb–object relations, and modifier attachment (exactly the kinds of errors a grammar checker looks for) map one-to-one onto labeled dependency edges, making rule-based or statistical error detection straightforward.
- **Language flexibility:** dependency parsing is more robust for languages/sentences with flexible word order, since it encodes grammatical function rather than fixed positional structure.

CFG trees remain valuable where the system needs to validate phrase-level well-formedness (e.g., “is this a legal noun phrase”) or to support constituency-based operations like phrase reordering. In practice, many systems even parse with a CFG/PCFG and then convert to dependencies via head rules. But for the stated goal — capturing relationships between words for grammar checking — dependency parsing is the more directly effective and efficient representation.

Q2. Top-Down Parsing vs. Earley Parsing – Dynamic, Incomplete Input

Scenario: A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing. Analyze the differences and reason which is more suitable for such dynamic input conditions.

Working Principles

Top-down parsing:

Starts from the start symbol S and repeatedly expands the leftmost non-terminal using grammar rules, trying to predict a structure that will eventually match the input left to right. When an expansion does not match, the parser backtracks and tries an alternative rule.

Earley parsing:

A chart-based dynamic-programming algorithm that processes input left to right, maintaining a set of “states” (partially matched rules) at every position via three operations — predict, scan, and complete. It never permanently commits to a wrong choice; instead, it keeps all viable partial analyses in the chart and reuses completed sub-derivations.

Key Differences

Aspect	Top-Down Parsing	Earley Parsing
Search strategy	Depth-first prediction with backtracking on failure	Breadth-first chart of all partial states, dynamic programming
Left recursion	Fails / infinite loop on left-recursive rules	Handled correctly
Reuse of computation	None – backtracking re-derives from scratch	Yes – chart stores and reuses completed states
Worst-case complexity	Exponential on ambiguous grammars	Polynomial – $O(n^3)$ worst case, near-linear for many grammars
Handling ambiguity	Each reading explored as a separate backtracked path	All readings kept compactly as a shared parse forest
Handling incomplete input	Poor – generally needs to commit to a full rule expansion	Natural – the chart at any prefix is a valid, well-defined partial parse

Reasoning – Suitability for Dynamic, Real-Time Input

- Incomplete input: because Earley’s chart at input position i already represents every grammatically valid partial analysis, the system can query it before the sentence is finished — useful for autocomplete-style or streaming input. Top-down parsing has no equivalent well-defined “halfway” state; a wrong early prediction only surfaces as failure much later, forcing a full backtrack.
- Ambiguous input: Earley’s packed parse forest avoids the combinatorial explosion that top-down backtracking suffers on ambiguous grammars, since it never re-explores structurally identical sub-parses.
- Predictable latency: Earley’s polynomial worst-case bound gives a real-time application predictable response times, whereas top-down parsing can spike to exponential time on pathological or highly ambiguous input.
- Conclusion: Earley parsing is more suitable for a real-time application handling incomplete or ambiguous input, because its dynamic-programming chart provides efficient, complete, and reusable partial analyses exactly where top-down parsing is weakest.

Q3. CFG vs. PCFG vs. Neural Parsing – Resolving Ambiguity

Scenario: An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques. Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.

The Ambiguity

The sentence has classic PP-attachment ambiguity: “with a telescope” can attach to the verb “saw” (she used a telescope as an instrument to see the man) or to the noun “man” (the man she saw was carrying/had a telescope). Both readings are syntactically legal.

How Each Approach Handles It

CFG:

Both attachments are valid derivations under the grammar, and CFG has no mechanism to prefer one over the other — it simply overgenerates both parse trees with equal standing, leaving the ambiguity fully unresolved.

PCFG:

Assigns a probability to every rule (learned from treebank frequencies), so the two competing parse trees now get different overall probabilities based on how often that attachment pattern occurred in training data. The parser can then select the maximum-probability tree. However, PCFG probabilities are typically estimated from local rule frequencies and largely ignore the specific lexical items involved (it does not “know” that telescopes are plausible instruments for seeing but implausible things for a man to be attached to in other contexts) unless lexicalized extensions are added.

Neural parsing:

Uses learned distributed representations (word embeddings, contextual embeddings from models like BERT/transformers) and can be trained end-to-end (e.g., neural dependency/constituency parsers, or a self-attention/transformer parser). It captures lexical-semantic plausibility — e.g., that “see ... with a telescope” is a common instrumental pattern — by learning from large corpora, and it uses the full sentence context (not just local rule statistics) to weigh the two readings. It generalizes better to attachment patterns and word combinations never seen verbatim in training.

Comparison Summary

Criterion	CFG	PCFG	Neural Parsing
Resolves ambiguity?	No – all parses equally valid	Yes – via rule probabilities	Yes – via learned contextual/semantic representations
Uses lexical/semantic context	No	Limited (mostly structural frequency)	Yes – rich contextual & semantic cues
Generalization to unseen sentences	N/A (no ranking)	Moderate	High
Interpretability	High	High	Lower (“black box”)
Data / compute requirement	None	Annotated treebank	Large training corpora, more compute

Which Is Most Effective for Real-World Applications

Neural parsing is the most effective overall for real-world use, because ambiguity like “saw ... with a telescope” is fundamentally a semantic-plausibility problem, not just a structural-frequency one — and neural models capture exactly that by learning from lexical and contextual patterns at scale, generalizing to novel sentences that a fixed PCFG grammar was never trained on. PCFG remains a reasonable middle ground when interpretability, low compute, or a small well-

defined domain matter, and CFG alone is unsuitable wherever genuine ambiguity resolution is required. In practice, many production systems use neural models (or neural-augmented statistical parsers) precisely because they combine the ambiguity-handling strength of probabilistic scoring with deep semantic context that plain PCFGs lack.

Q4. Feature Structures vs. Subcategorization Frames – Grammatical Constraints

Scenario: A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames. Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.

Working Principles

Feature structures:

Attribute–value matrices (AVMs) attached to grammar categories, e.g. NP[NUM:sg, PERS:3, GENDER:fem] or V[AGR:[NUM:sg, PERS:3]]. During parsing, unification checks that the feature values of combining constituents are compatible, which is the mechanism used to enforce agreement (subject–verb, determiner–noun) and other fine-grained grammatical properties like case or gender.

Subcategorization frames:

Lexical entries for predicates (mainly verbs) that specify the number and type of complements the predicate requires or allows — e.g. sleep[SUBCAT: –] (intransitive), eat[SUBCAT: NP] (transitive), give[SUBCAT: NP, NP] (ditransitive), or believe[SUBCAT: S'] (clausal complement). Their purpose is to constrain a verb’s argument structure (valency), rejecting sentences with missing or extra arguments.

Comparison

Aspect	Feature Structures	Subcategorization Frames
Phenomenon addressed	Agreement / concord (NUM, PERS, GENDER, CASE)	Argument structure / valency of predicates
Scope	General-purpose – any grammatical property, any category	Specific to predicates (mainly verbs)
Mechanism	Unification of attribute–value pairs across constituents	Lexical lookup of required complement pattern
Example error caught	“The dog run” (NUM mismatch)	“She gave the book” without recipient (missing argument)
Representation style	Declarative, compositional (values combine via unification)	Fixed lexical specification per predicate

Justification

The two mechanisms address different grammatical phenomena, so neither is a full substitute for the other: feature structures alone cannot detect that a verb is missing a required object, and a subcategorization frame alone cannot detect that a subject and verb disagree in number. For a grammar system that must handle both subject–verb agreement and argument structure (as the scenario requires), feature structures provide the more complete and extensible foundation — because in unification-based grammar formalisms (e.g., HPSG, LFG), subcategorization information is itself encoded as a SUBCAT feature inside the same feature-structure/AVM formalism. This lets one unification mechanism simultaneously

check agreement features and verify that a verb's complements match its subcategorization list, giving unified, compositional enforcement of grammatical correctness rather than maintaining two separate, disconnected mechanisms.

- Feature structures: broader, general, and can subsume subcategorization as one of their features – better long-term architectural choice.
- Subcategorization frames alone: more specialized, easier to hand-write for a small verb lexicon, but limited to argument structure and cannot express agreement.
- Conclusion: feature structures (with an embedded SUBCAT feature) provide better overall support for enforcing grammatical correctness, since they unify agreement and argument-structure checking within a single, extensible formalism.

Q5. Transition-Based vs. Graph-Based Parsing – Large-Scale Dependency Parsing

Scenario: A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing. Analyze the differences in decision-making and performance, and justify which is more suitable for large-scale applications.

Working Principles

Transition-based parsing:

Builds the dependency tree incrementally as a sequence of local decisions (e.g., shift, left-arc, right-arc in an arc-standard system), using a classifier (traditionally an SVM, now typically a neural network) to choose the next action based on the current parser configuration (stack/buffer state). The tree is built greedily (or with a small beam) in a single left-to-right pass.

Graph-based parsing:

Treats parsing as a global optimization problem: it scores every possible head-dependent edge to form a complete weighted graph over all words, then searches for the maximum spanning tree (e.g., via the Chu-Liu/Edmonds algorithm, or Eisner's algorithm for projective trees) that maximizes total edge score across the whole sentence.

Comparison of Decision-Making and Performance

Aspect	Transition-Based Parsing	Graph-Based Parsing
Decision-making	Local, incremental, greedy/beam-search choices	Global optimization over all possible edges at once
Optimality	Not guaranteed globally optimal; local errors can propagate	Globally optimal tree given the edge-scoring model
Time complexity	Linear – $O(n)$ per sentence (near-linear with beam search)	Higher – typically $O(n^2)$ or more for higher-order features
Long-distance dependencies	Weaker – error propagation over many steps	Stronger – scores all edges jointly, including long-range ones
Memory / compute at scale	Lower – scales well to very large corpora	Higher – more expensive per sentence
Parallelizability / throughput	High – simple sequential decisions, fast with neural scorers	Lower – spanning-tree search is comparatively costly

Justification – Suitability for Large-Scale Applications

- Speed and scalability: transition-based parsing's linear (or near-linear) time complexity per sentence makes it far more practical when processing millions of sentences, which is the defining requirement of a large-scale system.
- Accuracy trade-off is now smaller: modern neural transition-based parsers (using contextual embeddings to score transitions) close much of the accuracy gap that used to favor graph-based methods, particularly for local/short dependencies, while retaining their inherent speed advantage.
- Graph-based strength: graph-based parsing still gives higher accuracy on long-distance dependencies because it optimizes globally rather than committing to irreversible local decisions, but this comes at a real computational cost that is harder to justify at web-scale volumes.
- Conclusion: for a large-scale system where throughput and computational cost dominate, transition-based parsing is more suitable due to its linear-time, highly scalable decision process; graph-based parsing remains preferable only where maximum accuracy on individual sentences is worth the extra computational cost and dataset size is comparatively modest.